

Executive Summary

Cat-Dex is a full-stack, responsive web application engineered to gamify real-world feline encounters. Operating on a capture-and-collect mechanic, the system utilizes local artificial intelligence to process live camera feeds, identify subjects, isolate them from their backgrounds, and render them as digital 3D stickers. Built as an independent, self-directed engineering project, the entire application was **vibecoded**—architected, engineered, and debugged through highly iterative natural language AI collaboration—to demonstrate the successful integration of a modern React interface with a high-performance Python FastAPI backend.

System Architecture & Technologies

The application follows a decoupled client-server architecture communicating via REST APIs:

- **Frontend Interface:** Developed with React, Vite, and Tailwind CSS v4, handling dynamic UI states, device camera streams, and offline persistence.
- **Backend Engine:** Built on Python and FastAPI to manage asynchronous image processing and API routing.
- **AI & Computer Vision:** Utilizes local photo recognition models and the lightweight `rembg` (`u2netp` performance model) library to extract subjects and generate transparent base64 PNG assets locally.
- **Data Persistence:** Employs modular browser `localStorage` to cache user discoveries on the device for the "Yours" gallery grid.

Core Engineering Features

- **The Viewfinder:** A live camera feed overlaid with custom reticles and a staged capturing sequence ("Locking in...", "Peeling it off...") to mask asynchronous backend processing times.
- **Dynamic 3D Rendering:** Applies stacked CSS drop-shadow filters to extracted transparent images, creating the visual illusion of a physical, tactile sticker complete with rarity badges and dynamic data.
- **Imposter Guard:** An automated detection mechanism that intercepts non-feline subjects (e.g., cars, dogs), triggering a restricted "Not a cat!!" UI state that blocks the save function and alerts the user.

Development Challenges & Resolutions

The development lifecycle involved significant optimization to achieve a seamless, low-latency user experience. Initial testing with massive local inference models and cloud-based alternatives (such as the Gemini API) introduced processing bottlenecks, Python 3.14 SDK version conflicts, and 503 server capacity errors. Because the project was vibecoded, the architecture could be rapidly pivoted in real-time, allowing the system to be swiftly refactored to utilize a fully local, lightweight photo recognition pipeline. By pre-scaling image payloads to 500x500 pixels before inference, processing speeds were drastically improved. After resolving Vite import path errors and configuring a strict `.gitignore` shield, the optimized, API-key-free codebase was successfully deployed to version control.